Topic 2: Arrays

Jaber Taheri-Shakib

Research associate

Civil and Environmental Engineering

University of Waterloo| McMaster University

Creating a Matrix Manually

- Matrix notation for MatLab is a clear extension of vector notation.
  - Each value within a row is separated by a ***space*** or a ***comma***
  - Each new row is separated by a ***semi-colon***
- Beyond these, two clear rules govern matrix construction:
  1. Row construction has higher precedence than column construction.
  2. The number of values in each row must always be the same!

[ ]

```
% Option 1
m1 = [1 2 3;4 5 6; 7 8 9]

% Option 2
m2 = [1 2 3;
      4 5 6;
      7 8 9]
```

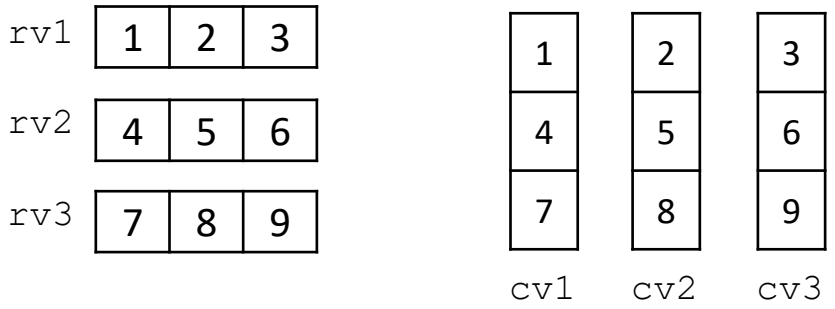
```
>> m1 =

     1     2     3
     4     5     6
     7     8     9

>> m2 =

     1     2     3
     4     5     6
     7     8     9
```

Creating a Matrix From Vectors

Matrices can be manually constructed in a variety of ways, including concatenation, use the colon operator, or a mixture of the two.

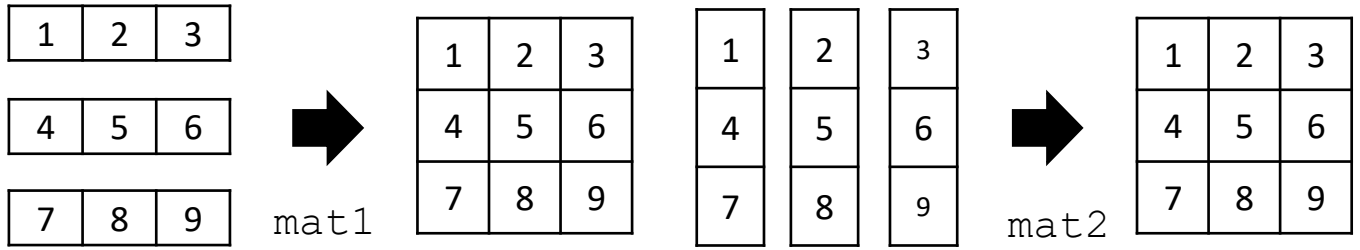
```
rv1 = [1 2 3];
rv2 = [4 5 6];
rv3 = [7 8 9];

mat1 = [rv1; rv2; rv3];

cv1 = [1; 4; 7];
cv2 = [2; 5; 8];
cv3 = [3; 6; 9];

mat2 = [cv1 cv2 cv3]
```

Matrix Concatenation



⚠: **There must <u>ALWAYS</u> be the same number of values in every row!!**

[1 2 3; 4 5 6]
[1 2 3; 4 5]



Some Matrix Generation Examples

```
sampMat1 = [1:4; 5:8; 9:12];

rv1 = [1 2 3];
rv2 = [4 5 6];
rv3 = [4 3 4 3]
rv4 = [7 6 7 6];

sampMat2 = [rv1;rv2];
sampMat3 = [rv1;4 5 6];
sampMat4 = [rv2; rv4];
sampMat5 = [rv1 rv3; rv2 rv4];
```

Name	Value
sampMat1	**[1 2 3 4; 5 6 7 8; 9 10 11 12]**
rv1	**[1 2 3]**
rv2	**[4 5 6]**
rv3	**[4 3 4 3];**
rv4	**[7 6 7 6];**
sampMat2	
sampMat3	
sampMat4	
sampMat5	

Functions that Create Matrices

Function	Description	Example	
		Script	Output Variable Value
`zeros(n)`	creates an *nxn* matrix of all zeros	`zeros(2)`	0 0 0 0
`zeros(n,m)`	creates an *nxm* matrix of all zeros	`zeros(2,3)`	0 0 0 0 0 0
`ones(n)`	creates an *nxn* matrix of all ones	`ones(2)`	1 1 1 1
`ones(n,m)`	creates an *nxm* matrix of all ones	`ones(2,3)`	1 1 1 1 1 1
`eye(n)`	creates an *nxn* identity matrix.	`eye(3)`	1 0 0 0 1 0 0 0 1
`eye(n,m)`	creates an *nxm* matrix, with the value of 1 on the main diagonal.	`eye(3,2)`	1 0 0 1 0 0
`rand(n)`	creates an *nxn* matrix of random reals on the interval (0,1)	`rand(2)`	0.8147 0.1270 0.9058 0.9134
`rand(n,m)`	creates an *nxm* matrix of random reals on the interval (0,1)	`rand(2,1)`	3 5 0 2
`randi([range], n)`	creates an *nxn* matrix of random integers in the specified range	`randi([0,5],2)`	
`randi([range], n,m)`	creates an *nxm* matrix of random integers in the specified range	`randi([0,5],2,3)`	0.5377 -2.2588 1.8339 0.8622
`randn(n)`	creates an *nxn* matrix of random samples from the standard normal distribution	`randn(2)`	
`randn(n,m)`	creates an *nxm* matrix of random samples from the standard normal distribution	`randn(2,1)`	

Accessing Matrix Elements

Each element in a matrix has an index associated with both the row and the column. To index, refer to the row first, then the column.

```
value = matrix(rowIndex, columnIndex)
```

```
myMat = [5 33 11;
         -4 2 12;
         1 47 -15];
```

Column Index

Row Index

	1	2	3
1	5	33	11
2	-4	2	12
3	1	47	-15

😂: horizontal second, vertical first

```
>> value = myMat(3,2)

value =

     47
```

Accessing More than One Element

Similarly to vectors, you can extract portions of a matrix for examination. MatLab accommodates this through index vectors on both the row and columns.

```
sub_matrix = matrix(row_indices, col_indices)
```

Through matrix indexing you can:

- Extract the contents of a row or column using the colon operator (:)
- Reference the last row or column of a matrix by combining the colon (:) and `end` operators.[A(end, :)A(:, end)]

A(1,2)
means row 1, column 2.

A([1 2], [2 3])

This means select rows 1 and 2, and columns 2 and 3.

A(:, 3)

means select all rows from column 3.

Example: Matrix Indexing

Suppose we have a randomly generated 4x4 matrix. How would we:

1. Extract values from the even rows?
2. Extract the last element of the matrix?
3. Extract the values in the 2x2 subset shown below?

```
>> randMat = randi([0,50], 4)
    randMat = randi([0,50],4,4)
randMat =

    41    32    48    48
    46     4    49    24
     6    14     8    40
    46    27    49     7
```

```
>> randMat(                    )

ans =
    46     4    49    24
    46    27    49     7

>> randMat(                    )

ans =

     7

>> randMat(                    )

ans =
     4    49
    14     8
```

Modifying Matrices

- An individual elements and groups of elements in a matrix can be modified and assigned new values.
- The dimension of the assigned values must be equivalent to the dimension of the subset being modified!
- Scalar assignments are an exception to the rule above. ***A scalar value can be assigned to a subset of any size***, and replaces each element in the subset.

```
matrix(row_indices, col_indices) = expression
```

```
A(2,3) = 100
```

```
A = [1 2 3;
     4 5 6;
     7 8 9]
```

```
A(1:2, 2:3) = [20 30; 50 60]
```

```
A(1:2, 2:3) = 0
```

Example: Matrix Modification

📖: A scalar value can be assigned to a subset of any size, and replaces each element in the subset.

```
>> mat = [1 2 3;4 5 6;7 8 9];
>> mat(1,end) = 30

mat =

>> mat(:,1) = [10;10;10]

mat =
```

… continued …

```
>> mat(end,:) = zeros(1,3)

mat =

>> mat(:,3) = mat(1,:)

mat =
```

… continued …

```
>> mat(2,:) = 1

mat =

>> mat([1 3], [1 3]) = 10

mat2 =
```

Example: Matrix Modification

... continued ...

Suppose we have a randomly generated 2x5 matrix. How would we:

1. Multiply the odd columns by 2?
2. Replace the even columns with the number 0?
3. Swap columns 2 and 3?

```
>> randMat = randi([20, 90], 2,5)

randMat =

    77    29    64    39    87
    84    84    26    58    88
```

```
>> randMat(              ) = randMat(              )


randMat =

   154    29   128    39   174
   168    84    52    58   176

>> randMat(              ) =

randMat =

   154     0   128     0   174
   168     0    52     0   176

>> randMat(              ) =

randMat =

   154   128     0     0   174
   168    52     0     0   176
```

Empty Vectors and Matrices

- An ***empty vector*** is a vector with no elements; an empty vector can be created using square brackets with nothing inside [ ]
- To **delete** an element from a vector, assign an empty vector to that element
- Delete an entire row or column from a matrix by assigning [ ]
  - Note: **<u>You cannot delete an individual element from a matrix</u>**

```
>> v1 = 2:6
v1 =

     2     3     4     5     6

>> v1(3) = []
v1 =

     2     3     5     6
```

```
>> m1 = [1 2 3;4 5 6;7 8 9]
m1 =

     1     2     3
     4     5     6
     7     8     9

>> m1(2,:) = []
m1 =

     1     2     3
     7     8     9
```

m1(:,2) = []
This means delete column 2 and all rows.

m1(2,2) = []

m1(2,2) = 0
or
m1(2,2) = NaN

Linear Indexing

- Arrays can also be indexed using one index value instead of two.
- MatLab will interpret this value by counting through the matrix column by column, top to bottom.

mat1(row, column)

1.2	5.7	3.1	7.5
3.8	2.2	2.0	4.9
4.3	2.8	8.7	9.1

Element Values

(1,1)	(1,2)	(1,3)	(1,4)
(2,1)	(2,2)	(2,3)	(2,4)
(3,1)	(3,2)	(3,3)	(3,4)

Row-Column Indexing

1	4	7	10
2	5	8	11
3	6	9	12

Linear Indexing

```
mat1 = [1.2 5.7 3.1 7.5;
        3.8 2.2 2.0 4.9;
        4.3 2.8 8.7 9.1];

val1 = mat1(2, 1)
val2 = mat1(2)
val3 = mat1(3, 4)
val4 = mat1(12)
vec1 = mat1(3:5)
```

Name	Value
val1	
val2	
val3	
val4	
vec1	

Printing Out Arrays

```
1   % Formatting some helpful output statements
2   fprintf('\n%10s%10s%10s\n', 'x','y','z');
3   fprintf('%10s%10s%10s\n', '----','----','----');
4
5   % Generating Input Vectors
6   x = [1 2 3 4 5];
7   y = [9 8 7 6 5];
8
9   % Calculating the Output
10  z = (x.^2-2*y).^3./(100*x) + ...
11      abs(x.*y)/(5*sqrt(10*x+y.^(7/3))) + ...
12      sin(x).*cos(y)/5;
13
14  % Correct Way to Format the Output Table
15  tableVals = [x;y;z];
16
17  % Incorrect Way to Format the Output Table
18  % tableVals = [x' y' z'];
19
20  % Printing out the results
21  fprintf('%10.1f%10.1f%10.2f\n', tableVals)
```

- `fprintf()` can be used with arrays, but the implementation is a bit tricky.
  - As with vectors, when the data arguments to `fprintf()` are array values, the function gets repeated until there are no more values left to print.
  - With multi-dimensional arrays, MatLab reads the data in linear indexing order (column-wise).

x1, y1, z1
x2, y2, z2
x3, y3, z3
x4, y4, z4

Correct

x	y	z
1.0	9.0	-48.96
2.0	8.0	-8.35
3.0	7.0	-0.08
4.0	6.0	0.33
5.0	5.0	7.02

tableVals = [x' y' z'];

Incorrect

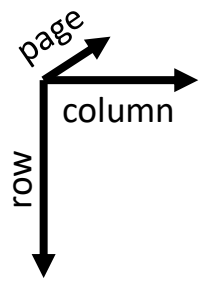
x	y	z
1.0	2.0	3.00
4.0	5.0	9.00
8.0	7.0	6.00
5.0	-49.0	-8.35
-0.1	0.3	7.02

Higher Dimensional Number Arrays

- MatLab has the capacity to deal with higher dimensional sets of information using higher dimensional arrays.
- ***No syntax exists for manual, single-line array generation***. Instead, a higher-dimensional array must first be generated and then values can be assigned at each index.
- The most common case is a 3D array, which has the dimensions `m x n x p`

```
mat3D = zeros(3, 4, 2);

mat1 = [1 2 3 4; 5 6 7 8; 9 10 11 12];
mat2 = [13 14 15 16; 17 18 19 20; 21 22 23 24];

mat3D(:,:,1) = mat1;
mat3D(:,:,2) = mat2;
```

array(row, column, page)



1	2	3	4
5	6	7	8
9	10	11	12

13	14	15	16
17	18	19	20
21	22	23	24

mat3D

1	2	3	4
5	6	7	8
9	10	11	12

mat1

13	14	15	16
17	18	19	20
21	22	23	24

mat2

3D Matrix Example … Continued

```
val1 = mat3D(1, 3, 1);

val2 = mat3D(1, 1, 2);

mats = mat3D(1:2, 1:2, 1);
```

Name	Value
mat3D	3x4x2 double
val1	
val2	
mats	

📖: Large matrices are not printed out in the Workspace.

1	2	3	4
5	6	7	8
9	10	11	12

13	14	15	16
17	18	19	20
21	22	23	24

Matrix Values

(1,1,1)	(1,2,1)	(1,3,1)	(1,4,1)
(2,1,1)	(2,2,1)	(2,3,1)	(2,4,1)
(3,1,1)	(3,2,1)	(3,3,1)	(3,4,1)

(1,1,2)	(1,2,2)	(1,3,2)	(1,4,2)
(2,1,2)	(2,2,2)	(2,3,2)	(2,4,2)
(3,1,2)	(3,2,2)	(3,3,2)	(3,4,2)

Subindex

Reshaping

📖: Values from A are given to output to B in linear indexing order.

`B = reshape(A,sz)` reshapes `A` using the size vector, `sz`, to define `size(B).sz` must contain at least 2 elements and the product of elements in `sz` must be the same as `numel(A)`.

```
vec = 3:14; % 12 elements

mat0 = reshape(vec, [2 6]);
mat1 = reshape(vec, [3 4]);
mat2 = reshape(mat1, [6 2]);
mat3 = reshape(mat2, [2 2 3]);
```

vec

3	4	5	6	7	8	9	10	11	12	13	14

mat0

mat1

mat2

mat3